ASSIGNMENT 1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50)

NAME: P DILLI BABU

REG NO: 192472284

Nature of Assignment: Individual

Duration: 1 Week

Submission Date:10-11-2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level) Mandatory Execution and GitHub Submission

GitHub Repository Link: https://github.com/dillibabu1122/DSA0115--C-/tree/main/AT1

TASK 1 – Understand the Problem and Identify the Objects

1.1 Object-oriented analysis

Object / Class	Important data	Behaviour / responsibility
SmartCard	cardId, balancePaise, passengerId	Check remaining balance, accept recharge, approve and deduct fare without violating the Rs 50 minimum.
Passenger	passengerId, name, passengerType	Identify passenger category and determine whether a concession policy applies.
Journey	journeyId, source, destination, zone, hour, weekend flag	Store trip details and provide the inputs required for fare calculation.
Zone	zone value / zone name	Represent Inner, Middle, Outer; later Airport. Zone determines base fare.
FareTable	base fares, surcharge percentage, discount rules	Return the base fare and apply the applicable fare policy consistently.

SmartCard is an object because each passenger card has its own balance and operations. Keeping the balance private prevents unrelated code from setting it to a negative value. Journey groups trip-related information instead of passing many unrelated parameters. FareTable centralizes prices so a fare revision is made in one place. Passenger separates passenger identity and concession eligibility from card balance. Zone is represented by a strongly typed enumeration.

1.2 Procedural style versus OOP

A procedural solution would use variables such as balance, zone, hour and fare, together with free functions such as calculateFare(), canTravel() and deductFare(). All functions must agree on the same variables and rules. In an OOP solution, SmartCard owns the balance, Journey owns trip details, and FareTable owns pricing. Related data and behaviour are grouped, and private data can only be changed through validated methods.

1.3 Four OOP principles

- Encapsulation: SmartCard keeps balancePaise private and exposes recharge(), canTravel() and deduct(). This protects the minimum-balance rule.
- Abstraction: A gate calls fareTable.calculate(journey); it does not need to know the internal arithmetic or discount sequence.
- Inheritance: StudentPassenger and SeniorPassenger may derive from Passenger if they share common identity data but have different concession eligibility rules.

- Polymorphism: A virtual concessionRate() or calculateConcession() method can be overridden by student and senior passenger types. The actual rate must be supplied by the metro policy.

1.4 Rules and assumptions

- After a successful journey, card balance must be at least 5,000 paise (Rs 50.00).
- A denied journey must not change the card balance.
- Base fare is Rs 10, Rs 20 or Rs 30 for Inner, Middle or Outer.
- Peak surcharge is 20% from 08:00 inclusive to 10:00 exclusive, and 17:00 inclusive to 20:00 exclusive.
- All money calculations use integer paise; the percentage calculation uses integer arithmetic.
- Invalid zones, invalid hours and negative recharge amounts must be rejected.
- Concession eligibility and rate require an official policy value; no rate is assumed in this report.

1.5 Alternative concept and complexity

In a procedural version, global variables or a Card structure would hold the balance and free functions would calculate fares and deduct amounts. Both approaches take O(1) time for one fare calculation and O(1) memory per card. The OOP version is easier to maintain because fare data and card validation are centralized. If the minimum balance changes, a centralized constant and SmartCard validation method are updated once. In a procedural design, every function that performs or checks deductions must be reviewed, increasing the risk of inconsistent changes.

Conclusion: Use OOP for the metro gate because it groups responsibilities and protects critical balance rules without changing the constant-time cost of processing a journey.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly

2.1 Five kinds of C++ tokens

Token kind	Examples in this assignment	Purpose
Keywords	class, private, public, if, switch, return, constexpr	Reserved words with language-defined meaning.
Identifiers	SmartCard, balancePaise, calculateFare, peak	Names created by the programmer.
Constants / literals	5000, 20, true, 'A'	Fixed values used in expressions.
Operators	+ , -, *, /, %, ==, &&, ?:, =	Perform arithmetic, comparison, logic or assignment.
Punctuators	{ }, ( ), ; , , [ ]	Structure declarations, statements, calls and arrays.

2.2 Constants

#define is a preprocessor text substitution and has no type or normal C++ scope. const creates a typed object that cannot be modified after initialization. constexpr requires a compile-time constant expression where possible and is suitable for fixed fare values. constexpr is preferred because the compiler checks types and can use the values during compilation.

```
constexpr int INNER_FARE_PAISE = 1000;
constexpr int MIDDLE_FARE_PAISE = 2000;
constexpr int OUTER_FARE_PAISE = 3000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
```

enum class Zone { Inner, Middle, Outer }; is safer than a plain enum because its values are scoped (Zone::Inner) and do not implicitly convert to int. It avoids accidental mixing with unrelated integer values.

2.3 Identifier naming

- Use meaningful names such as balancePaise and peakSurchargePercent.
- Use PascalCase for class and type names, such as SmartCard and FareTable.
- Use UPPER_SNAKE_CASE for fixed constants, such as MIN_BALANCE_PAISE.
- Identifiers cannot contain spaces, cannot start with a digit and cannot be a C++ keyword.
- Avoid unclear names such as x1 or temp when a descriptive name is possible.

2.4 Alternative: macros

With #define INNER_FARE 1000, the preprocessor replaces matching text before compilation. A macro has no type checking, can affect unrelated code through textual substitution, and does not obey normal namespace or block scope. constexpr and enum class provide type checking and safer scope. A macro generally occupies no storage; a constexpr value also commonly needs no runtime storage when used as a compile-time constant. Neither approach meaningfully changes fare-calculation time. The typed constexpr/enum class design is safer and easier to debug.

TASK 3 – Choose Data Types and Check Type Compatibility

3.1 Data type selection

Value	Recommended type	Reason
Fare / card balance	long long (paise)	Exact integer arithmetic; avoids decimal rounding.
Surcharge percentage	constexpr int	Small whole-number percentage.
Hour / zone index	int / enum class Zone	Hour is 0–23; zone is better represented by an enum.
Journey count / network total	long long	Can exceed the signed int range

		when counting paise.
Peak flag / eligibility	bool	Only true or false is required.
Single character input	char	Stores one character.
Display amount	double only if needed for display	Do not use as the authoritative money value.
Input type inference	auto	Useful when the initializer makes the type clear.

3.2 Why integer paise?

Rs 10.25 is stored as 1,025 paise. Addition, subtraction and percentage calculations can be performed using integers. Binary floating-point cannot represent many decimal fractions exactly; repeated operations can produce values such as 0.1 + 0.2 not being exactly 0.3. Integer paise preserves exact currency values as long as the chosen integer type does not overflow.

3.3 Type compatibility

- When int and double appear in an arithmetic expression, the int is normally converted to double before the operation.
- When bool is used in arithmetic, false becomes 0 and true becomes 1 after integral promotion.
- An expression such as 7 / 2 uses integer division and produces 3; assigning that result to double gives 3.0, not 3.5.
- Use static_cast<double>(7) / 2 when floating-point division is intended. For money, prefer integer paise and explicit integer rounding rules.

3.4 Ranges and daily collection

Typical 32-bit int range from <climits> is -2,147,483,648 to 2,147,483,647 (INT_MIN to INT_MAX). A 64-bit long long has a much larger range. Exact sizes are implementation-dependent and should be printed using sizeof on the student's compiler.

```
#include <climits>
#include <iostream>
int main() {
    std::cout << "sizeof(int) = " << sizeof(int) << " bytes\n";
    std::cout << "sizeof(long long) = " << sizeof(long long) << " bytes\n";
    std::cout << "sizeof(double) = " << sizeof(double) << " bytes\n";
    std::cout << "INT_MIN = " << INT_MIN << ", INT_MAX = " << INT_MAX << '\n';
}
```

For a conservative maximum fare of Rs 36.00 (3,600 paise, the Rs 30 fare plus 20% peak surcharge), 3,000,000 journeys collect Rs 108,000,000 = 10,800,000,000 paise. The rupee total fits a 32-bit signed int, but the paise total exceeds INT_MAX. Therefore, use long long for aggregate paise.

3.5 Alternative: double rupees

A double-based implementation stores each amount in approximately 8 bytes on common systems, and a long long also commonly uses 8 bytes. Both perform one addition in O(1) time. However, double can accumulate rounding error, while integer paise is exact for whole paise. The error depends on the sequence and magnitude of values; it should be measured by comparing the double sum with an integer-paise reference over the actual test data. Integer paise is preferred for financial correctness.

TASK 4 – Control Implicit Conversions

4.1 Conversion points

- int division: 7 / 2 is evaluated as integer division before assignment to double.
- Integer-to-double: assigning an int result to double converts the already-computed integer result.
- char promotion: ch + 1 promotes ch to int and adds one to its character code.
- int-to-bool: assigning 5 to bool produces true; printing bool normally displays 1, while std::boolalpha displays true.
- Signed/unsigned comparison: comparing -1 (int) with 1 (unsigned int) converts the signed operand to unsigned; -1 becomes a very large unsigned value.
- Narrowing: int x{3.7}; is rejected because list initialization prevents a potentially value-losing conversion.

4.2 Exact predicted output

```
3 3.5
C 68
1 true
s >= u
```

Explanation: c receives 3 because a / b is integer division. d is 3.5 because a is explicitly converted to double before division. 'A' + 2 produces the character C; ch + 1 is promoted to int and prints 68. bool flag = 5 converts nonzero to true; the first print is 1 and std::boolalpha changes the second to true. In s < u, -1 is converted to unsigned and becomes greater than 1, so the conditional expression prints s >= u.

4.3 Safer code and alternative

```
int a{7};
int b{2};
double c = static_cast<double>(a / b); // still 3.0: division happened first
double d = static_cast<double>(a) / static_cast<double>(b); // 3.5
char ch = static_cast<char>('A' + 2);
bool flag = (5 != 0);
```

Explicit casts document intentional conversions, while brace initialization rejects narrowing. They generally do not change runtime time or memory because casts are compile-time type conversions. They help the compiler catch narrowing mistakes during compilation. They do not automatically make a logically unsafe signed/unsigned comparison correct; compare compatible types or validate before conversion.

TASK 5 – Write Correct Expressions with the Right Precedence

5.1 Fare expressions

```
bool peak = (hour >= 8 && hour < 10) ||
            (hour >= 17 && hour < 20);
long long farePaise = baseFarePaise;
if (peak) {
   farePaise += farePaise * PEAK_SURCHARGE_PERCENT / 100;
}
```

The parentheses make the two peak windows clear. Relational operators are evaluated before logical AND (&&), and logical AND before logical OR (||). The unparenthesized expression in the question is equivalent because && has higher precedence than ||, but explicit parentheses improve readability and reduce maintenance errors.

5.2 Exact predicted output

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Line 1: multiplication has higher precedence than addition, so 5 + (3 × 2) = 11; parentheses force (5 + 3) × 2 = 16. Line 2: x++ yields the old value 5 and then increments x to 6; ++y increments y from 3 to 4 and yields 4; z = 5 + 4 = 9. Line 3: x > y selects 6; x % 4 is 2 and then multiplied by 2 gives 4. Line 4: integer division truncates toward zero; -10 / 3 is -3 and the remainder is -1. Line 5: i takes 0, 3, 6 and 9; their sum is 18.

5.3 Operators and alternative lookup table

- x++ returns the old value and increments afterwards; ++x increments first and returns the new value.
- Do not write i = i++: modifying and reading the same scalar in an unsequenced expression can produce undefined behaviour in relevant forms; use i++ as a standalone statement.
- For integer division, the quotient truncates toward zero. The remainder has the sign of the dividend: -10 % 3 is -1.
- The ?: operator selects one of two expressions based on a condition. Compound assignment such as += updates the left operand.

A bool peak[24] table would precompute which hours are peak and use peak[hour]. Both approaches are O(1) per check. The table uses 24 bool elements (commonly 24 bytes, but sizeof(bool) should be checked). It can reduce repeated comparisons, but the range expression is more self-documenting and does not require table initialization. Always validate hour is in [0,23] before indexing.

TASK 6 – Design the Control Structures

6.1 Pseudocode

```
ALGORITHM CalculateMetroFare
```

```
INPUT zone, hour, cardBalance
IF hour < 0 OR hour > 23 THEN
   OUTPUT "Invalid hour"
   STOP
END IF
IF zone is not a supported zone THEN
   OUTPUT "Invalid zone"
   STOP
END IF

SWITCH zone
   CASE Inner:  baseFare <- 1000
   CASE Middle: baseFare <- 2000
   CASE Outer:  baseFare <- 3000
   DEFAULT: OUTPUT "Invalid zone"; STOP
END SWITCH

peak <- (hour >= 8 AND hour < 10) OR
      (hour >= 17 AND hour < 20)
fare <- baseFare
IF peak THEN
   fare <- fare + (fare * 20 DIV 100)
END IF

IF cardBalance - fare >= 5000 THEN
   cardBalance <- cardBalance - fare
   OUTPUT fare and cardBalance
ELSE
   OUTPUT "Denied (low balance)"
END IF
END ALGORITHM
```

6.2 Control structure selection

- if–else is suitable for validating hour, checking peak periods and enforcing the minimum balance.
- switch is suitable for selecting a base fare from a finite set of zones. A default case handles an unsupported enum value.
- The conditional operator is useful for a short two-way selection, but nested ?: expressions make fare rules harder to read.
- A range-based for loop is suitable for processing a known array or container of journeys. A while loop is useful when journeys arrive continuously until a stop condition. A conventional for loop is suitable when an index or known count is needed.
- continue skips an invalid journey and proceeds to the next one. break exits a loop or switch when the current operation must stop. A switch default branch handles unmatched cases.

6.3 Completed code (TODO 1 and TODO 2)

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak) {
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default: return -1; // invalid zone
    }
    if (peak) {
        base += base * PEAK_SURCHARGE_PERCENT / 100;
    }
    return base;
}

bool canTravel(long long balance, long long f) {
    return f >= 0 && balance >= f &&
           balance - f >= MIN_BALANCE_PAISE;
}

int main() {
    long long balance = 10000;
    struct Trip { Zone zone; bool peak; };
    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}
    };
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.'
                      << std::setw(2) << std::setfill('0') << f % 100
                      << std::setfill(' ');
```

```
        } else {
            std::cout << "Denied (low balance)";
        }
        std::cout << " balance Rs " << balance / 100 << '\n';
    }
}
```

Expected output:

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

At exactly 10:00, hour < 10 is false, so it is off-peak. At exactly 20:00, hour < 20 is false, so it is off-peak. At exactly 08:00 and 17:00, the inclusive lower bounds make the journey peak.

TASK 7 – From Procedural to Object-Oriented

7.1 Procedural version

```
constexpr long long MIN_REMAINING = 5000;
bool canTravelProcedural(long long balance, long long fare) {
    return fare >= 0 && balance >= fare &&
           balance - fare >= MIN_REMAINING;
}
bool deductProcedural(long long& balance, long long fare) {
    if (!canTravelProcedural(balance, fare)) return false;
    balance -= fare;
    return true;
}
```

7.2 Object-oriented version

```
class SmartCard {
private:
    long long balancePaise;
    static constexpr long long MIN_REMAINING_PAISE = 5000;
public:
    explicit SmartCard(long long initialBalance)
        : balancePaise(initialBalance >= 0 ? initialBalance : 0) {}

    long long balance() const { return balancePaise; }

    bool canTravel(long long farePaise) const {
        return farePaise >= 0 && balancePaise >= farePaise &&
               balancePaise - farePaise >= MIN_REMAINING_PAISE;
    }
```

```
    bool deduct(long long farePaise) {
        if (!canTravel(farePaise)) return false;
        balancePaise -= farePaise;
        return true;
    }

    bool recharge(long long amountPaise) {
        if (amountPaise <= 0) return false;
        balancePaise += amountPaise;
        return true;
    }
};
```

The class prevents a negative balance because outside code cannot directly modify balancePaise. The only deduction operation checks the fare and the remaining Rs 50 condition before changing the value. Both procedural and OOP versions take O(1) time per check/deduction and O(1) storage per card.

7.3 Public balance alternative

If balance were public, any part of the program could execute card.balancePaise = -100 or subtract a fare without checking the minimum. Time and memory would not materially change, but encapsulation would be lost and the non-negative/minimum-balance invariant could be broken. A private member with validated methods is the safer design.

TASK 8 – Read and Print Data Safely

8.1 Safe numeric input using cin

```
#include <iostream>
#include <limits>
bool readHour(int& hour) {
    while (true) {
        std::cout << "Enter hour (0-23): ";
        if (std::cin >> hour) {
            if (hour >= 0 && hour <= 23) return true;
            std::cout << "Hour must be between 0 and 23.\n";
        } else {
            std::cout << "Invalid input. Enter a number.\n";
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            return false;
        }
    }
}
```

When extraction fails, cin.fail() becomes true and the stream must be recovered. cin.clear() resets the error flags. cin.ignore(max, '\n') discards the invalid line so the same characters are not read repeatedly.

8.2 Money formatting

```
long long amountPaise = 12345;
std::cout << "Rs " << amountPaise / 100 << '.'
          << std::setw(2) << std::setfill('0')
          << amountPaise % 100 << std::setfill(' ') << '\n';
```

This prints Rs 123.45. setw(2) sets the minimum width for the next output field, and setfill('0') pads the paise part with zero. fixed and setprecision(2) are useful for display-only double values, but the authoritative money value should remain integer paise.

8.3 getline and stoi alternative

An alternative is to read a full line using getline(), then call stoi() inside try/catch. Parsing a line of length L takes O(L) time and requires O(L) buffer space. cin >> int is convenient and does not require storing the entire line, but recovery requires clearing the stream and discarding the bad input. getline plus stoi provides explicit exception handling and can validate trailing characters, but uses a line buffer. Both are suitable when implemented carefully.

TASK 9 – Implement and Test the Fare Logic

9.1 Test policy

For the test table, the 20% peak surcharge is calculated in integer paise. The peak windows are [08:00,10:00) and [17:00,20:00). The tests below use the three original zones. Unknown zone, negative recharge and non-numeric hour input must be rejected.

Test	Zone / input	Hour	Expected fare	Expected result
T1	Inner	07:00	Rs 10.00	Off-peak
T2	Inner	08:00	Rs 12.00	Peak boundary inclusive
T3	Middle	10:00	Rs 20.00	Off-peak boundary
T4	Outer	17:00	Rs 36.00	Peak boundary inclusive
T5	Outer	20:00	Rs 30.00	Off-peak boundary
T6	Middle	19:59	Rs 24.00	Peak
T7	Invalid zone	12:00	N/A	Reject input
T8	Recharge = -500 paise	N/A	N/A	Reject recharge
T9	Hour = abc	N/A	N/A	cin failure; clear and discard input

T10	Inner, balance Rs 60.00	Off-peak	Rs 10.00	Accepted; exactly Rs 50 remains

9.2 Five-journey hand calculation

Journey	Zone / time	Fare	Running total
1	Inner, 07:30	Rs 10.00	Rs 10.00
2	Middle, 08:30	Rs 24.00	Rs 34.00
3	Outer, 12:00	Rs 30.00	Rs 64.00
4	Inner, 17:15	Rs 12.00	Rs 76.00
5	Middle, 20:00	Rs 20.00	Rs 96.00

The five fares total Rs 96.00 (9,600 paise). The exact remaining card balance depends on the starting balance and the minimum-balance rule; the fare total itself is Rs 96.00.

9.3 Alternative table-driven testing

A table of t test cases processed in a loop takes O(t) time and O(t) storage for the test table. It is easier to add a boundary case by appending one row rather than writing a new hand-coded condition. A test table also makes expected-versus-actual comparison systematic. The test table above is the expected result; after execution, fill in actual results and mark each PASS or FAIL.

TASK 10 – Compare Expectations with Execution

10.1 Network-scale arithmetic

The question does not specify the number of journeys in each fare band, concession percentages or the peak/off-peak mix. Therefore, an exact real-world daily collection cannot be derived from the question alone. To provide a reproducible upper-fare illustration, assume every journey is charged the maximum original fare: Rs 30 plus 20% peak surcharge = Rs 36.00 = 3,600 paise.

Journeys/day	Total at Rs 36 each	Total in paise	Does signed 32-bit int fit paise total?
20,000	Rs 720,000	72,000,000	Yes
500,000	Rs 18,000,000	1,800,000,000	Yes, narrowly
3,000,000	Rs 108,000,000	10,800,000,000	No

At 3,000,000 journeys, the rupee total Rs 108,000,000 fits a typical 32-bit signed int, but the paise total 10,800,000,000 exceeds INT_MAX. A long long is required for the aggregate in paise. A double can represent the scale but may not represent every decimal currency fraction exactly.

10.2 Double versus integer paise experiment

```
#include <iostream>
#include <iomanip>
int main() {
    const int n = 3000000;
    long long exactPaise = 0;
    double rupees = 0.0;
    for (int i = 0; i < n; ++i) {
        exactPaise += 3600; // Rs 36.00
        rupees += 36.00;
    }
    std::cout << "Integer paise: " << exactPaise << '\n';
    std::cout << std::fixed << std::setprecision(2)
              << "Double rupees: " << rupees << '\n';
    std::cout << "Exact rupees: " << exactPaise / 100.0 << '\n';
}
```

Expected exact result: 10,800,000,000 paise = Rs 108,000,000.00. On common systems the double output for this repeated whole-number addition will normally display the same two-decimal result, but that does not prove floating-point is exact for all decimal fare values. Compare the values at higher precision to observe any representation difference.

10.3 Compiler warnings and alternative numeric types

Compile with g++ -std=c++17 -Wall -Wextra -Wconversion -pedantic source.cpp. Warnings depend on compiler version and exact code. Typical warnings include signed/unsigned comparison, possible narrowing conversion and unused variables. Fix the cause rather than suppressing the warning.

Accumulator	At 3,000,000 × 3,600 paise	Assessment
int (32-bit)	10,800,000,000	Signed overflow; result is not reliable.
long long	10,800,000,000	Fits comfortably; recommended.
double	10,800,000,000.0 approximately	Large enough, but binary floating-point is not exact for all decimal values.

Each addition is O(1) for all three types and each accumulator is O(1) space. The type choice affects range and precision, not asymptotic complexity.

TASK 11 – Recommend and Defend the Final Design

- Use enum class Zone for type-safe zone values and constexpr constants or a central FareTable for fixed policy values.
- Store all balances, fares and collections as long long integer paise.

- Use SmartCard with a private balance and validated recharge(), canTravel() and deduct() methods.
- Use Journey to hold zone, hour, date/weekend and journey identifiers.
- Keep fare calculation in FareTable or a dedicated fare-policy class so that the gate does not duplicate pricing rules.
- Validate every input before calculating or indexing arrays. Reject unknown zones, invalid hours and negative recharge amounts.
- Use unit tests for all peak boundaries, minimum-balance boundary, invalid inputs and large aggregate totals.

A fare revision should be made in one central fare configuration or table, followed by recompilation and the complete regression test suite. The main limitation of compile-time constants is that a fare change requires a new build and deployment. A configurable fare file can reduce this operational burden, but it must be validated, versioned and protected from unauthorized modification.

For the whole network, each journey remains O(1) time and each card uses O(1) state. Processing N journeys takes O(N) total time. long long paise supports the illustrated 3,000,000-journey maximum-fare total. Production deployment should additionally consider concurrency, transaction logging, duplicate tap prevention, refunds, security, and official concession/pass rules.

11.1 Configuration-file alternative

If z zone fares are loaded from a configuration file at gate startup, reading and validating the fare table takes O(z) time and O(z) memory. After loading, each fare lookup can remain O(1). This avoids recompiling for every fare revision, but the program must handle missing, malformed, outdated or tampered files. For a small fixed table, compile-time constants are simple and safe; for frequent policy revisions, a signed and validated configuration file is more flexible.

TASK 12 – DAY-4 DESIGN CHALLENGE: Re-evaluate and Adapt the Design

12A. Impact analysis

The new requirements add Airport (Rs 60), a 10% weekend discount that cannot combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two selected zones.

Existing design element	Required change
Zone enum	Add Zone::Airport. This changes the enum and every switch or zone validation list.
Fare constants/table	Add Airport fare 6,000 paise; preferably replace scattered switch values with one indexed fare table.
Peak surcharge expression	Keep 20% surcharge only on weekdays; weekend discount takes precedence and must not combine with peak surcharge.
Journey	Add dayType/weekend flag and date or service-period information.

SmartCard	Retain private balance and validation; add pass entitlement handling without bypassing card security.
New Pass class	Store two selected zones, validity month and pass status; allow unlimited journeys only in the selected zones during validity.

12B. Evaluate original design

The original switch-based fare logic can support Airport by adding another case, but repeated switches become harder to maintain as zones increase. The SmartCard class can be retained because its private balance and validated operations remain useful. However, the fare policy needs modification to support mutually exclusive weekend and peak rules, and the pass feature needs a separate abstraction.

Worked examples: (1) Airport weekday peak fare = Rs 60 + 20% = Rs 72.00. (2) Airport weekend fare = Rs 60 − 10% = Rs 54.00, even if the clock hour falls in a normal peak window, because weekend discount and peak surcharge do not combine. (3) A valid monthly pass covering Inner and Middle makes a journey in either selected zone fare-free during the pass validity; Outer and Airport remain chargeable unless included.

12C. Adapted design

Decision: MODIFY the original OOP design. Retain SmartCard and Journey, replace the base-fare switch with a central indexed fare table, add a weekend rule and introduce a MonthlyPass class. A full redesign is unnecessary because the original class boundaries already separate balance, journey and fare responsibilities.

```
#include <array>
#include <iostream>
#include <optional>
#include <string>

enum class Zone { Inner = 0, Middle = 1, Outer = 2, Airport = 3 };

constexpr std::array<long long, 4> BASE_FARE_PAISE{
   1000, 2000, 3000, 6000
};
constexpr int PEAK_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
constexpr long long MIN_BALANCE_PAISE = 5000;

class MonthlyPass {
   bool active;
   int firstZone;
   int secondZone;
```

```
public:
    MonthlyPass(bool isActive, Zone a, Zone b)
        : active(isActive), firstZone(static_cast<int>(a)),
          secondZone(static_cast<int>(b)) {}

    bool covers(Zone z) const {
        int value = static_cast<int>(z);
        return active && (value == firstZone || value == secondZone);
    }
};

long long calculateFare(Zone zone, int hour, bool weekend,
                        const MonthlyPass& pass) {
    int index = static_cast<int>(zone);
    if (index < 0 || index >= static_cast<int>(BASE_FARE_PAISE.size()) ||
        hour < 0 || hour > 23) return -1;

    if (pass.covers(zone)) return 0; // unlimited covered-zone travel

    long long amount = BASE_FARE_PAISE[index];

    if (weekend) {
        // Weekend discount replaces, and never combines with, peak surcharge.
        amount -= amount * WEEKEND_DISCOUNT_PERCENT / 100;
    } else {
        bool peak = (hour >= 8 && hour < 10) ||
                    (hour >= 17 && hour < 20);
        if (peak) amount += amount * PEAK_PERCENT / 100;
    }
    return amount;
}

class SmartCard {
    long long balance;
public:
    explicit SmartCard(long long initial) : balance(initial >= 0 ? initial : 0) {}
    bool pay(long long fare) {
        if (fare < 0 || balance < fare ||
            balance - fare < MIN_BALANCE_PAISE) return false;
        balance -= fare;
        return true;
    }
    long long getBalance() const { return balance; }
```

```
};

int main() {
    MonthlyPass pass(true, Zone::Inner, Zone::Middle);
    SmartCard card(20000);

    long long airportWeekdayPeak =
        calculateFare(Zone::Airport, 9, false, pass);
    long long airportWeekend =
        calculateFare(Zone::Airport, 18, true, pass);
    long long innerPassFare =
        calculateFare(Zone::Inner, 9, false, pass);

    std::cout << "Airport weekday peak: Rs "
              << airportWeekdayPeak / 100 << "."
              << (airportWeekdayPeak % 100 < 10 ? "0" : "")
              << airportWeekdayPeak % 100 << '\n';
    std::cout << "Airport weekend: Rs "
              << airportWeekend / 100 << "."
              << (airportWeekend % 100 < 10 ? "0" : "")
              << airportWeekend % 100 << '\n';
    std::cout << "Inner with pass: Rs "
              << innerPassFare / 100 << "."
              << (innerPassFare % 100 < 10 ? "0" : "")
              << innerPassFare % 100 << '\n';
}
```

Expected example output:

```
Airport weekday peak: Rs 72.00
Airport weekend: Rs 54.00
Inner with pass: Rs 0.00
```

Implementation note: A production monthly pass must also validate its start/end dates, passenger/card ownership, selected zones and renewal/expiry status. The sample code demonstrates the core two-zone entitlement rule; it does not implement calendar-month date validation.

12D. Evidence-based defence

Test	Expected calculation	Expected fare
Airport weekday at 09:00	Rs 60 × 1.20	Rs 72.00
Airport weekend at 18:00	Rs 60 × 0.90; no peak surcharge	Rs 54.00
Inner with active Inner/Middle pass	Covered by pass	Rs 0.00
Outer with active Inner/Middle	Not covered; normal fare	Rs 30.00

pass, weekday off-peak		
Middle after pass expiry	Pass inactive; normal fare	Rs 20.00 off-peak

The modified design preserves O(1) fare lookup and O(1) pass-zone checking because the fare table and two-zone membership check have fixed size. It adds a small amount of memory for pass data. The array is easier to extend with a fourth zone than maintaining multiple switch statements. The separate MonthlyPass class keeps pass eligibility independent from card balance. The design should be retained with these modifications and validated with boundary, pass-expiry and non-combination tests.

APPENDIX – Complete Revised Fare Program (Original Three-Zone System)

The following standalone program integrates zone validation, peak-hour calculation, integer-paise fares, minimum-balance protection and safe input. It is suitable as a base for the original three-zone implementation.

```
#include <iomanip>
#include <iostream>
#include <limits>
#include <string>

enum class Zone { Inner, Middle, Outer };

constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

bool isPeak(int hour) {
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);
}

bool readInteger(const std::string& prompt, int& value) {
    std::cout << prompt;
    if (std::cin >> value) return true;
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    return false;
}

bool readZone(Zone& zone) {
    int choice;
    if (!readInteger("Zone (1=Inner, 2=Middle, 3=Outer): ", choice))
        return false;
```

```
    switch (choice) {
        case 1: zone = Zone::Inner; return true;
        case 2: zone = Zone::Middle; return true;
        case 3: zone = Zone::Outer; return true;
        default: return false;
    }
}

long long calculateFare(Zone zone, int hour) {
    if (hour < 0 || hour > 23) return -1;
    long long amount = 0;
    switch (zone) {
        case Zone::Inner: amount = BASE_PAISE[0]; break;
        case Zone::Middle: amount = BASE_PAISE[1]; break;
        case Zone::Outer: amount = BASE_PAISE[2]; break;
        default: return -1;
    }
    if (isPeak(hour))
        amount += amount * PEAK_SURCHARGE_PERCENT / 100;
    return amount;
}

class SmartCard {
private:
    long long balancePaise;
public:
    explicit SmartCard(long long initial)
        : balancePaise(initial >= 0 ? initial : 0) {}

    bool canTravel(long long farePaise) const {
        return farePaise >= 0 && balancePaise >= farePaise &&
               balancePaise - farePaise >= MIN_BALANCE_PAISE;
    }

    bool deduct(long long farePaise) {
        if (!canTravel(farePaise)) return false;
        balancePaise -= farePaise;
        return true;
    }

    long long getBalance() const { return balancePaise; }
};
```

```
void printMoney(long long paise) {
    std::cout << "Rs " << paise / 100 << '.'
              << std::setw(2) << std::setfill('0') << paise % 100
              << std::setfill(' ');
}

int main() {
    SmartCard card(10000); // Rs 100.00
    int hour;
    Zone zone;

    if (!readZone(zone)) {
        std::cout << "Invalid zone or input.\n";
        return 1;
    }
    if (!readInteger("Enter hour (0-23): ", hour) ||
        hour < 0 || hour > 23) {
        std::cout << "Invalid hour or input.\n";
        return 1;
    }

    long long fare = calculateFare(zone, hour);
    if (fare < 0) {
        std::cout << "Invalid journey.\n";
        return 1;
    }

    if (card.deduct(fare)) {
        std::cout << "Journey approved. Fare: ";
        printMoney(fare);
        std::cout << ", Remaining balance: ";
        printMoney(card.getBalance());
        std::cout << '\n';
    } else {
        std::cout << "Journey denied. Minimum Rs 50 must remain.\n";
    }
    return 0;
}
```

Conclusion

The recommended metro fare system uses strongly typed zones, centralized fare rules, integer paise, validated input and an encapsulated SmartCard class. It prevents a journey from reducing the balance below Rs 50, treats peak-hour boundaries consistently, and scales to millions of journeys when long

long is used for aggregate paise. The Day-4 requirements are best handled by modifying the original OOP design: add Airport to a central fare table, apply weekend discount as a mutually exclusive rule, and introduce a separate MonthlyPass class.